MEDIXAI: EXPLAINABLE AI BASED MEDICAL EXPERT SYSTEM USING RULE BASED REASONING

A PROJECT REPORT

Submitted by

NISHANTH ELANGO RAJAN (241901075)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "MEDIXAI - Explainable AI Based Medical Expert System Using Rule Based Reasoning" is the bonafide work of "Nishanth Elango Rajan (241901075)" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. P. Tamilselvi, Ph.D.,

HEAD OF THE DEPARTMENT

PROFESSOR

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai - 602 105

SIGNATURE

Kumaresan

SUPERVISOR

ASSISTANT PROFESSOR (SG)

Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai - 602 105

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ABSTRACT

MEDIXAI is an explainable medical expert-system web application designed to demonstrate classical artificial intelligence and modern full-stack web programming through explicit knowledge representation and rule-based reasoning. The project addresses the limitations of simple symptom lookup by presenting ranked hypotheses together with the matched weights, required premises, and clinical precautions behind each result. The system uses deterministic rules and excludes black-box machine learning.

The foundational baseline contains 38 disease cards, 104 distinct symptom labels, and 228 weighted mappings (retained as the offline browser LocalStorage fallback), while the full-stack SQLite database (medixai.db) expands coverage to 98 clinical diseases across 12 categories and 397 distinct symptoms with 639 weighted links. The engine normalizes input, ranks candidate rules by weighted symptom coverage, applies a 0.60x multiplier when required premises are missing, and supports interactive backward premise checks.

The full-stack architecture combines a semantic HTML5, CSS3 Grid/Flexbox, and vanilla ES6 single-page application (index.html) with a Python HTTP REST server (app.py) and normalized five-table SQLite persistence. Interactive browser workflows support live symptom search, expandable symptom analytics, dynamic CRUD expansion of diseases, symptoms, and contradiction pairs, and persistent evaluation logs.

The proposed consultation workflow separates input correction, candidate evaluation and presentation. Observations are validated before inference, and each result retains the matched weight, total card weight, missing required symptoms and penalty status. This shared evidence record is intended to keep the browser explanation, optional history entry and exported report consistent. A published knowledge version provides a stable context for reproducing a saved calculation.

The evaluation plan combines controlled arithmetic examples with proposed checks for conflicting observations, unknown identifiers, explanation fidelity and access control. Future work includes explicit present, absent and unknown states, focused follow-up questions and separately tested formal reasoning demonstrations. The project emphasizes reproducibility and a clear account of implemented behavior as the basis for further development.

Keywords: MEDIXAI, Rule-Based Expert System, Classical Artificial Intelligence, Explainable AI, Forward Chaining, Medical Decision Support.

ACKNOWLEDGEMENT

We express our sincere thanks to our beloved and honourable chairman **Mr. S. Meganathan B.E., F.I.E.,** and the chairperson **Dr. M. Thangam Meganathan Ph.D.,** for their timely support and encouragement.

We are greatly indebted to our respected and honourable principal **Dr. S.N. Murugesan M.E., Ph.D.,** for his able support and guidance.

We express our sincere thanks to **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security).**

We also extend our sincere and hearty thanks to our internal guide **Mr. Kumaresan,** for his valuable guidance and motivation during the completion of this project.

Our sincere thanks to our family members, friends and other staff members of computer science engineering.

NISHANTH ELANGO RAJAN (241901075)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
1	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Objectives	2
	1.4 Scope	2
	1.5 Security Requirements and Security Considerations	3
	1.6 SDG Alignment	3
2	**LITERATURE REVIEW**	**4**
	2.1 Existing System with Security Approaches	4
	2.2 Research Gaps	5
	2.3 Proposed System	6
	2.3.1 Overview of the Proposed System	6
	2.3.2 Objectives of the Proposed System	6
	2.3.3 Advantages of Proposed System	6

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
3	**SYSTEM DESIGN**	**7**
	3.1 System Architecture	7
	3.2 Module Design	9
	3.3 Data Flow and UML Diagrams	11
	3.4 Threat Model and Attack Surface	16
	3.5 Security Controls	17
4	**SYSTEM DESCRIPTION**	**18**
	4.1 Data Collection	18
	4.2 Data Preprocessing	19
	4.2.1 Algorithms and Methodology	20
	4.2.2 Results and Discussion	23
5	**CONCLUSION**	**26**
	REFERENCES	**27**
	APPLICATION SCREENSHOTS	**28**
	APPENDIX A Knowledge Base Inventory	32
	APPENDIX B Proposed Data Dictionary	34
	APPENDIX C Inference Pseudocode	36
	APPENDIX D Implementation and Acceptance Plan	38

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

MEDIXAI is an academic expert system that demonstrates how explicit knowledge and logical reasoning can turn symptom observations into ranked disease hypotheses. Its purpose is to make each result inspectable: a reader should be able to identify the matched symptoms, missing conditions and calculation behind the ranking. The project uses classical artificial intelligence through explicit rules, knowledge representation and transparent reasoning.

The project has two levels of development. A Python command-line prototype illustrates symptom matching, weighted scoring, required-symptom verification and explanation generation. A proposed web application separates the user interface, inference engine and knowledge repository so that the same reasoning procedures can operate on an updated knowledge base. Machine learning, deep learning and trained prediction models are outside the scope.

1.2 PROBLEM STATEMENT

A simple symptom-to-disease lookup can hide why an answer was selected and can fail when only part of a rule is satisfied. MEDIXAI addresses the engineering problem of ranking several supported hypotheses while exposing their evidence and limitations. It must also manage inconsistent input and protect sensitive information if consultation history is introduced. A ranking is an educational output, not a verified medical diagnosis.

Reasoning workflow and project context

A consultation can be understood as a sequence of evidence-handling steps. The user supplies observations, the system converts them into canonical symptom identifiers, and the inference procedure compares those identifiers with the stored disease cards. The output must preserve the connection between an observation and its contribution to a score. This sequence gives students a practical way to connect knowledge representation with the result shown on screen.

The main engineering challenge is to keep that explanation consistent as the application grows. A change to a symptom weight should be visible in the knowledge version and in any newly generated result. An earlier saved consultation should retain its original evidence. These requirements make reproducibility, maintainability and clear communication central to the project, alongside the ranking calculation itself.

The report therefore treats the prototype as a worked foundation for an educational application.

1.3 OBJECTIVES

Table 1.1 links the project objectives to acceptance evidence. The primary goal is an explainable expert system with outputs traceable to facts, rules and weights. Keeping domain knowledge separate from inference makes changes reviewable and avoids rewriting application logic for every new disease.

Table 1.1 Project objectives and acceptance evidence

Objective	Evidence required
Represent domain knowledge	Structured disease cards, symptom identifiers and required-symptom sets.
Rank multiple hypotheses	Reproducible scores and ordering for fixed inputs.
Explain each result	Matched and missing evidence, penalty status and rule identifiers.
Verify logical support	Goal-directed checks of the selected hypothesis.
Handle inconsistent input	Visible conflicts and a defined correction workflow.
Support secure expansion	Documented access controls, schema and test cases.

1.4 SCOPE

The project covers requirements, literature review, system design, inspection of the prototype and deterministic scoring examples. The planned application includes guest access, registration, a symptom interface, ranked results, explanations, consultation history, PDF export and knowledge administration. It uses Python, Flask, SQLite and HTML/CSS/JavaScript. These application features remain design commitments until implemented and tested.

Clinical deployment, prescription generation, medication dosages, hospital integration and claims of diagnostic accuracy are excluded. No patient dataset is used to train or validate the supplied prototype.

Scope boundaries and acceptance

Acceptance should be based on observable software behavior. For a fixed observation set and knowledge snapshot, the engine should produce the same scores, the same required-symptom status and a consistent candidate order. A reviewer should be able to recalculate a displayed value from the recorded weights without relying on hidden application state.

The browser interface should distinguish a completed calculation from a saved consultation. A guest may inspect a result without creating an account, while persistent history requires an explicit storage action.

1.5 SECURITY REQUIREMENTS AND SECURITY CONSIDERATIONS

The security design must protect consultation confidentiality, rule integrity and service availability. Guest consultations should avoid persistent personal data. Stored histories and generated reports must be accessible only to their owners or an explicitly authorized role. Knowledge-base edits need separate authorization because a changed weight or rule can alter every subsequent result.

Input must be restricted to known symptom identifiers and bounded request sizes. The application must treat rule content as data, use parameterized database queries and avoid executing imported expressions. Explanations should report what the engine actually evaluated. Authentication, session protection, authorization and logging controls are proposed for the web version; the command-line listing does not implement them.

1.6 SDG ALIGNMENT

Table 1.2 SDG alignment

S. No.	SDG No. and goal	Relevant target	Project contribution and alignment
1	SDG 3 Good Health and Well-being	3.8 Access to essential health services	Indirect educational alignment through understandable health-information workflows. No improvement in health outcomes or access has been measured.
2	SDG 4 Quality Education	4.4 Technical and vocational skills	Direct academic alignment through knowledge engineering, logical inference, database design and secure software development.

Table 1.2 maps the project to SDG 3 and SDG 4 through understandable health-information workflows and technical learning. These intended contributions do not establish achievement of the UN indicators.

Applying the requirements

Security requirements should be attached to individual workflows. A history request needs an owner check, a knowledge edit needs an administrative permission check, and an export needs the same access decision as the result it represents. Keeping these decisions at the server prevents the interface from becoming the sole enforcement point.

The educational contribution can be evaluated through student tasks such as tracing a ranking, finding a missing requirement and explaining why two cards overlap. Such exercises connect the project to technical learning outcomes without treating the application as evidence of improved health-service delivery.

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

The literature establishes rule-based consultation, goal-directed inference and explanation as longstanding AI methods. MEDIXAI applies these ideas in a small, inspectable academic application. Its contribution should therefore be judged by implementation clarity and evaluation rather than by claiming that expert systems or explainability are new.

Table 2.1 Literature and design relevance

Work	Relevant contribution	Use in MEDIXAI
Russell and Norvig AIMA, 4th edition	Logical agents, propositional inference, forward and backward chaining.	Terminology and distinction between matching, entailment and proof.
Buchanan and Shortliffe, editors MYCIN, 1984	Medical expert-system rules, uncertainty and explanation.	Historical foundation; no transfer of clinical validation.
van Melle, Shortliffe and Buchanan EMYCIN, 1984	A reusable consultation engine separated from domain knowledge.	Basis for the proposed expert-system shell.
OWASP ASVS 5.0.0, 2025	Requirements for application security verification.	Reference for web security requirements and tests.

Table 2.1 relates the AI literature to reasoning methods and the ASVS standard to security requirements. The historical expert-system works support the design foundations; they are not current web-security standards.

Implications for the design

The separation between a knowledge base and an inference engine is particularly useful for this project. Students can inspect a card, change a reviewed weight in a controlled copy, and observe the effect on ranking without changing the procedure that evaluates every card. This makes the source of a changed result easier to identify.

Explanation also needs a precise meaning. In MEDIXAI, it should identify selected evidence, missing required evidence, the denominator used for coverage and any applied penalty. A persuasive paragraph generated independently of those values would not establish explanation fidelity. The design consequently favors an evidence record that can be rendered consistently in the browser and in an exported report.

The literature review guides terminology and architecture.

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Table 2.2 compares the initial five-disease example with the expanded prototype and planned application. The initial example requires all listed symptoms, then uses partial matching and a combined yes/no question. The expanded prototype adds weighted ranking, missing-required-symptom checks and selected contradiction pairs.

Table 2.2 Comparison of the project baselines

Dimension	Initial example	Expanded prototype and proposed application
Knowledge representation	Five symptom lists in Python.	38 weighted cards in the listing; database storage proposed.
Partial evidence	Match count and one follow-up prompt.	Weighted ranking and explicit required-symptom penalty.
Explanation	Final label and treatment text.	Matched/missing evidence; detailed rule trace proposed.
Input consistency	No dedicated conflict list.	Three pair checks; correction gate proposed.
Security	Local execution only.	Web authorization and private history proposed.

2.2 RESEARCH GAPS

The prototype lacks a formal rule agenda, multi-step proof traces, calibrated scores, source provenance for individual rules and validated medical ground truth. It also does not distinguish an explicitly absent symptom from one that was never asked about. These gaps motivate engineering and evaluation work; they do not establish superiority over commercial symptom checkers.

Priorities arising from the gaps

The first improvement should be a stable representation of observations. Present, absent and unknown states need distinct values so that unanswered questions do not silently become negative evidence. The next priority is a result contract that records exactly which weights and requirements were evaluated for each candidate.

A later formal reasoning module should be introduced with its own rule syntax, stopping conditions and proof records. Testing it separately from the weighted matcher would make its contribution assessable. Security and usability evaluation should likewise use explicit scenarios, rather than treating a larger knowledge base or an attractive interface as evidence that the identified gaps have been resolved.

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system is a modular web application with an independent inference service. The service accepts validated observations, reads a versioned knowledge snapshot and returns candidate results with a structured explanation. Flask routes coordinate the request, SQLite stores the knowledge and optional history, and the browser displays the outcome.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

The application will expose several hypotheses, show the contribution of each selected symptom and ask focused follow-up questions when important evidence is unknown. A knowledge administrator will maintain disease cards and rule metadata through controlled forms. Saved reports will retain the knowledge version so that an earlier result remains understandable after later edits.

2.3.3 ADVANTAGES OF PROPOSED SYSTEM

Separating the layers allows the interface and database to change without rewriting the scoring procedure. Structured explanations make defects easier to locate, and explicit missing evidence helps students understand why a hypothesis lacks support. The planned novelty is the integration of transparent ranking, traceable reasoning and maintainable knowledge in one educational workflow.

These advantages are design expectations. Reuse in another domain must be demonstrated with a second knowledge base, and usability must be measured with users. Better clinical accuracy cannot be inferred from a larger disease list or a more polished interface.

Proposed consultation experience

The interaction should begin with a clear symptom-selection form and a short explanation of how observations will be used. Before inference, the server checks the submitted identifiers and returns any conflicts for correction. Once the input is accepted, the result view presents a ranked list together with matched evidence and missing requirements for each displayed candidate.

Follow-up questions should be optional, focused and bounded. A response updates the current observation set and triggers a fresh calculation against the same knowledge snapshot. If the user decides to save, the application stores the resulting evidence and version together. The export then presents that same snapshot, so differences between an on-screen result and a downloaded report can be detected during testing.

This workflow turns explainability into a visible part of the consultation rather than a separate summary appended after scoring.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The proposed architecture uses a browser presentation layer, a Flask application layer, an inference service and a SQLite persistence layer. The boundary between the browser and server is untrusted. The inference service receives canonical facts rather than raw form text.

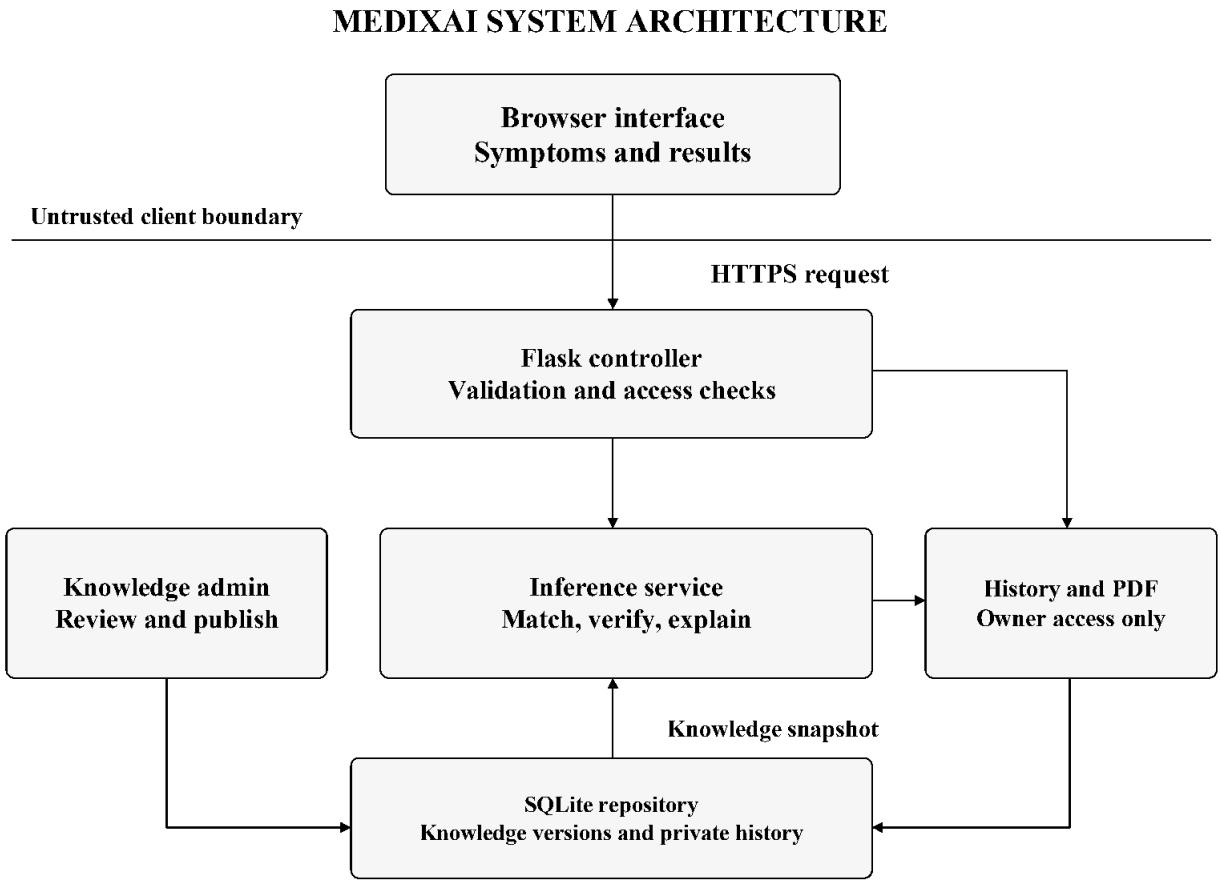


Figure 3.1 MEDIXAI system architecture

Figure 3.1 separates the browser, controller, inference service and repository. Each consultation uses an immutable knowledge snapshot, and one result object supplies the web explanation, optional history and PDF export. Knowledge editing follows a separate administrative path. This is the target web design; the original prototype is a single Python class.

Layer interaction

Each layer should expose a narrow contract: validated observations enter the inference service, and structured evidence leaves it. Database access belongs in the repository, while request validation and owner checks belong in the application workflow. This division makes failures easier to isolate and allows the same calculation to support several presentation formats.

3.1 SYSTEM ARCHITECTURE

Table 3.1 Hardware requirements

Component	Proposed development configuration
Processor	General-purpose dual-core processor or better.
Memory	4 GB RAM minimum planning allowance; 8 GB preferred for development tools.
Storage	At least 1 GB free for source, dependencies and demonstration outputs.
Graphics	No dedicated GPU required by the rule-based design.
Network	Local development can use loopback; hosting requires a secured network connection.

Table 3.2 Software requirements

Component	Role
Python 3	Core logic and application code.
Flask	Planned HTTP routes and server-side application workflow.
SQLite	Planned relational knowledge and consultation storage.
HTML, CSS and JavaScript	Browser forms and results presentation.
ReportLab	Planned PDF consultation report generation.
Chart.js	Optional visualization of rule-match scores.
Git and an editor	Version control, development and review.

Table 3.1 lists planned hardware allowances; Table 3.2 identifies the software components. These are planning assumptions, not measured minimum requirements. Dependencies must be pinned and tested; inference needs no external AI API or model-training service.

Development and deployment preparation

A reproducible setup should record the interpreter version, installed dependencies and database initialization steps. A small demonstration knowledge snapshot and fixed sample inputs should accompany the source so that another student can verify the calculation before enabling account or history features.

Hosting introduces configuration choices that local execution does not exercise. The implementation plan should therefore distinguish a local demonstration from a network-accessible application and record the settings used for each verified build.

3.2 MODULE DESIGN

Table 3.3 defines the five core AI modules, their inputs and their outputs. Their interfaces should return structured data, allowing the command-line demonstration and web application to share the same inference engine.

Table 3.3 Core module responsibilities

Module	Input	Output
Symptom collection	Entered or selected symptoms.	Canonical observation set and input errors.
Knowledge base	Reviewed disease cards and rule records.	Versioned knowledge snapshot.
Inference engine	Facts and a knowledge snapshot.	Ranked candidates and required-symptom status.
Consistency checking	Positive and negative observations.	Conflict pairs and correction requests.
Explanation and output	Candidate record and evidence trace.	Readable explanation and educational guidance.

A result record should carry the disease identifier, score, matched symptoms, missing required symptoms, total possible weight, applied penalty and knowledge version. This contract makes a score independently checkable. Rule identifiers should refer to actual rules evaluated; a displayed identifier alone does not demonstrate that a formal proof was executed.

The current explanation routine describes only the highest-ranked candidate in detail. The proposed interface extends the same evidence view to each displayed candidate.

Module contracts and error handling

Validation errors should be returned as structured messages that identify the affected input without exposing internal exceptions. The inference module should receive an already validated knowledge snapshot, but it should still reject an invalid scoring denominator rather than return a misleading value.

The explanation module should consume the candidate record directly. It should not repeat the scoring procedure in a second implementation, because two copies could diverge after a maintenance change. Returning all evidence in one shared record also makes it possible to compare the browser, stored history and PDF output during acceptance testing.

Clear contracts allow each module to be reviewed with small controlled examples before the full application is assembled.

3.2 MODULE DESIGN

The supporting modules provide the planned application workflow. Authentication manages registered and guest access. Consultation routes pass validated observations to the engine. History and report modules use the same result snapshot. Administrative routes manage knowledge separately from personal consultation data.

Table 3.4 Planned database tables

Table	Purpose
users	Account identity, password hash and role.
disease_categories	Disease grouping labels.
symptom_categories	Symptom grouping labels.
diseases	Disease name, description and status metadata.
symptoms	Canonical symptom codes and display names.
disease_symptoms	Weighted many-to-many symptom mapping.
treatments	Reviewed educational guidance text.
precautions	Reviewed caution and referral text.
rules	Rule identity, conditions and priority metadata.
diagnosis_history	Owned consultation and result snapshot.
history_symptoms	Observations linked to a saved consultation.

Table 3.4 organizes the eleven planned tables for knowledge, accounts and consultation history. Validation, review and snapshot fields support the design without implying that the database is already implemented. Appendix B defines proposed constraints.

Persistence workflow

Knowledge maintenance and consultation storage have different lifecycles. A reviewed knowledge change creates a published version for later consultations. Saving a consultation records the observations and complete result produced by the version already in use. These operations should not share an implicit update path.

The repository should group related writes into a transaction so that a history record cannot be presented as complete when its observations have not been stored. A failed write should leave the user with a clear outcome and preserve the current displayed result for a deliberate retry.

Retrieval should return a complete snapshot rather than rebuild an old consultation from the active knowledge base. This keeps historical evidence stable after a later publication.

3.3 DATA FLOW AND UML DIAGRAMS

The data flow separates consultation evidence from administrative knowledge changes. A registered user may explicitly save a consultation. Guest requests are processed without persistent history by default.

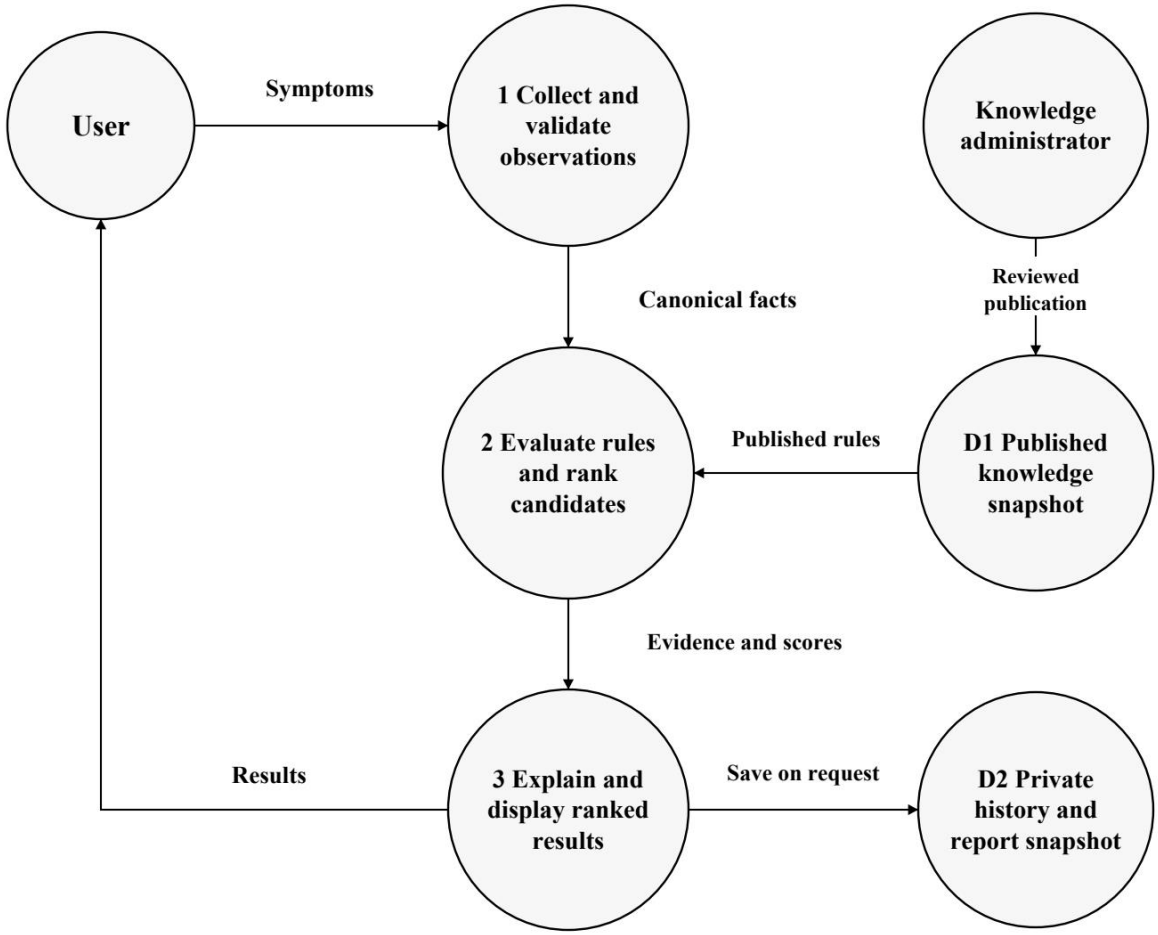


Figure 3.2 MEDIXAI consultation data flow

Figure 3.2 traces observations through collection, inference and result display using circular nodes. Published knowledge versions prevent an administrative edit from changing an in-progress consultation. Optional saving and export use the current or stored result snapshot and require the same ownership checks as the results page.

Reading the data flow

The numbered processes describe transformations of data rather than individual web pages. Collection produces canonical observations; evaluation produces candidates and evidence; presentation converts that evidence into a readable result. A private history write is a separate optional branch, and knowledge publication is controlled independently of the consultation request.

3.3 DATA FLOW AND UML DIAGRAMS

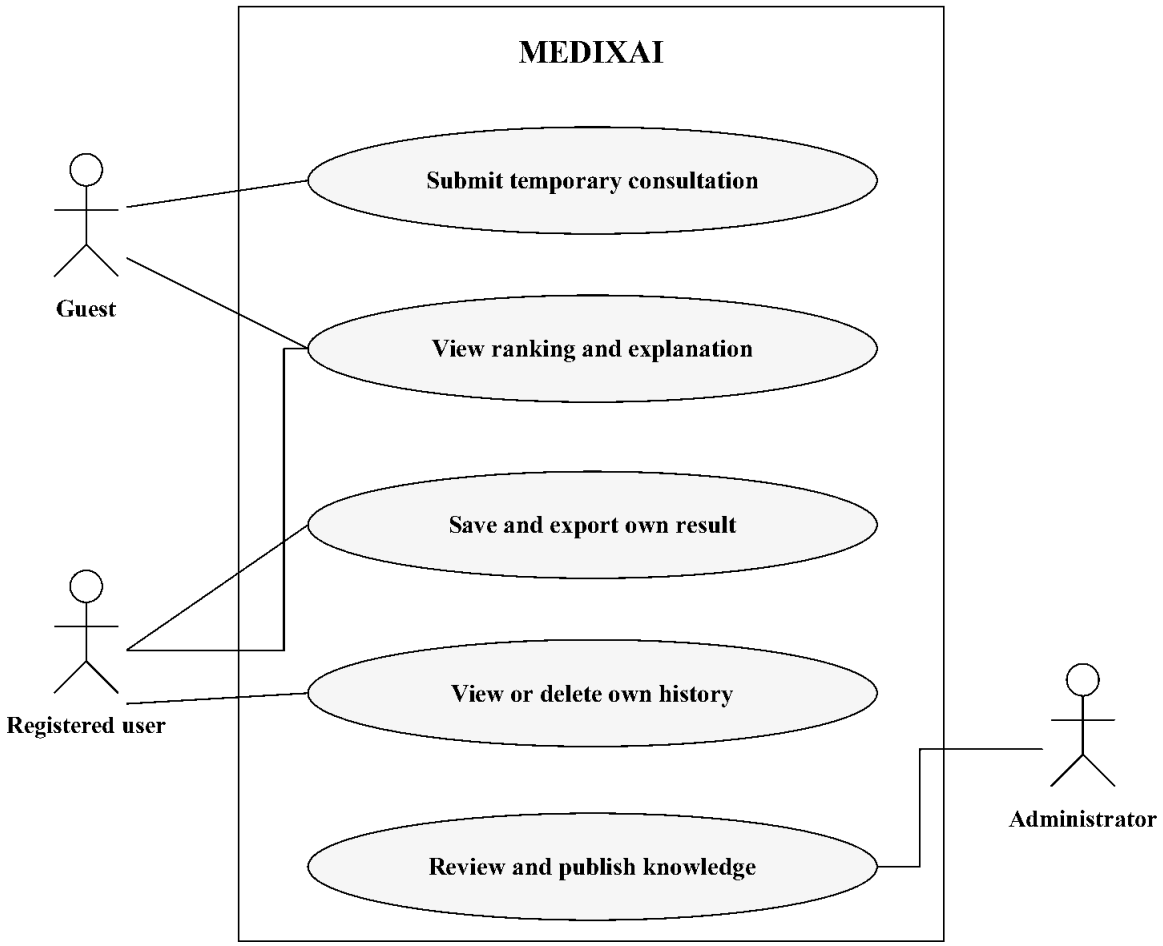


Figure 3.3 MEDIXAI use case diagram

Figure 3.3 shows that guests can submit temporary consultations and inspect explanations. Registered users can also save, view and delete their own history. Administrators maintain knowledge records, but that role does not automatically permit access to every personal consultation.

Each history or report request must be authorized at the server. Hiding a navigation link is insufficient because a user could directly request a resource identifier. The user interface should explain when data will be saved and provide a clear way to end the consultation.

Actor responsibilities

Permissions should be expressed in terms of actions and resources. Viewing an explanation for the current temporary consultation does not require the same permission as retrieving an older saved report. A registered account can manage its own stored consultations, while knowledge publication belongs to the administrative workflow.

The use case boundary helps identify acceptance scenarios.

3.3 DATA FLOW AND UML DIAGRAMS

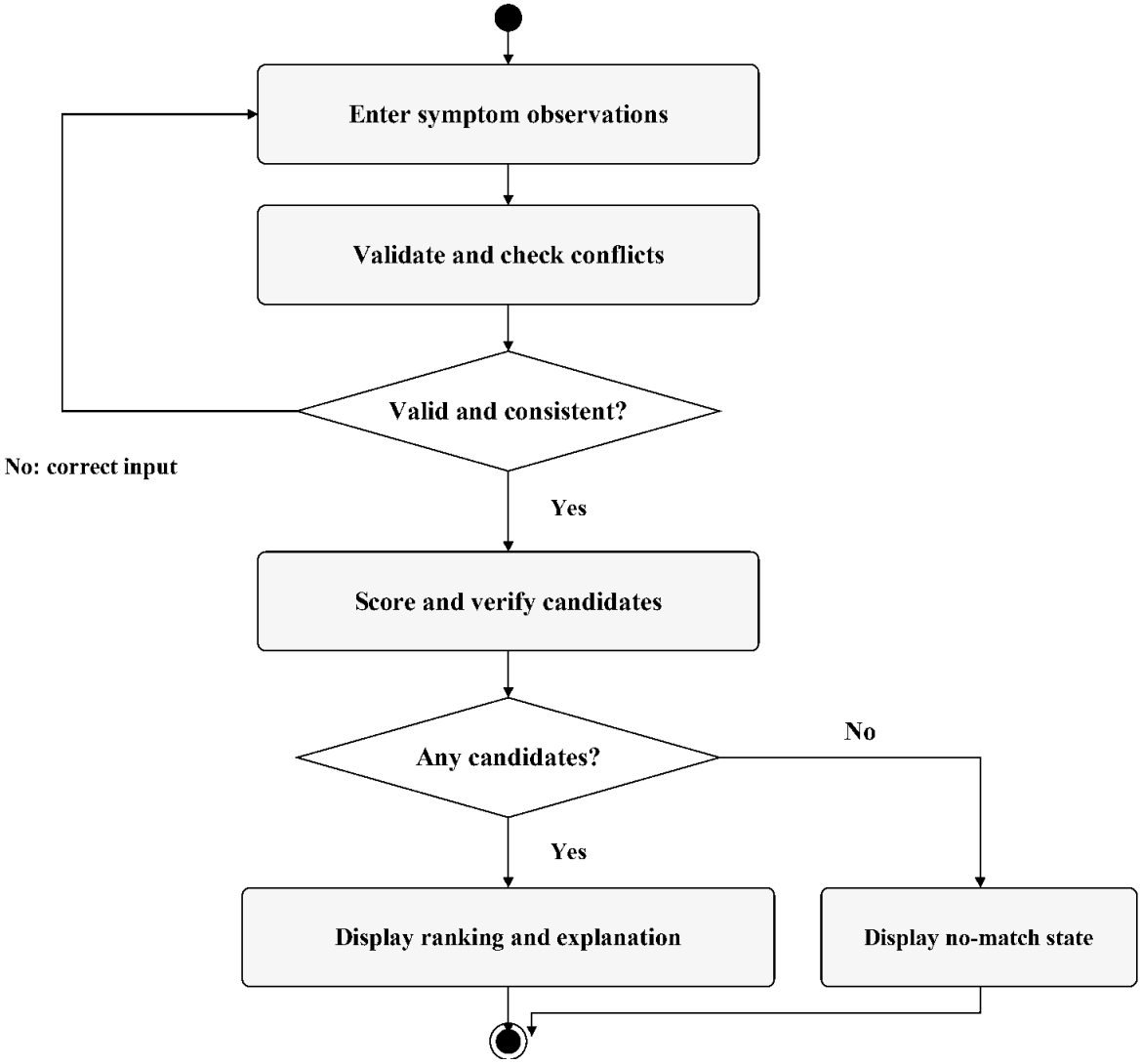


Figure 3.4 MEDIXAI consultation activity diagram

Figure 3.4 returns invalid or conflicting observations to the input stage. Unknown answers remain unknown rather than becoming absent symptoms. The proposed correction gate differs from the original prototype, which prints a contradiction warning and continues to calculate rankings.

Decision points and termination

The first decision checks whether the observations are suitable for evaluation. A correction response should identify the specific conflicting or unknown identifier and retain the other valid selections. This avoids forcing the user to restart the consultation after a recoverable input problem.

The second decision distinguishes a ranked result from a no-match outcome. Both are valid end states of the calculation and should explain what happened.

3.3 DATA FLOW AND UML DIAGRAMS

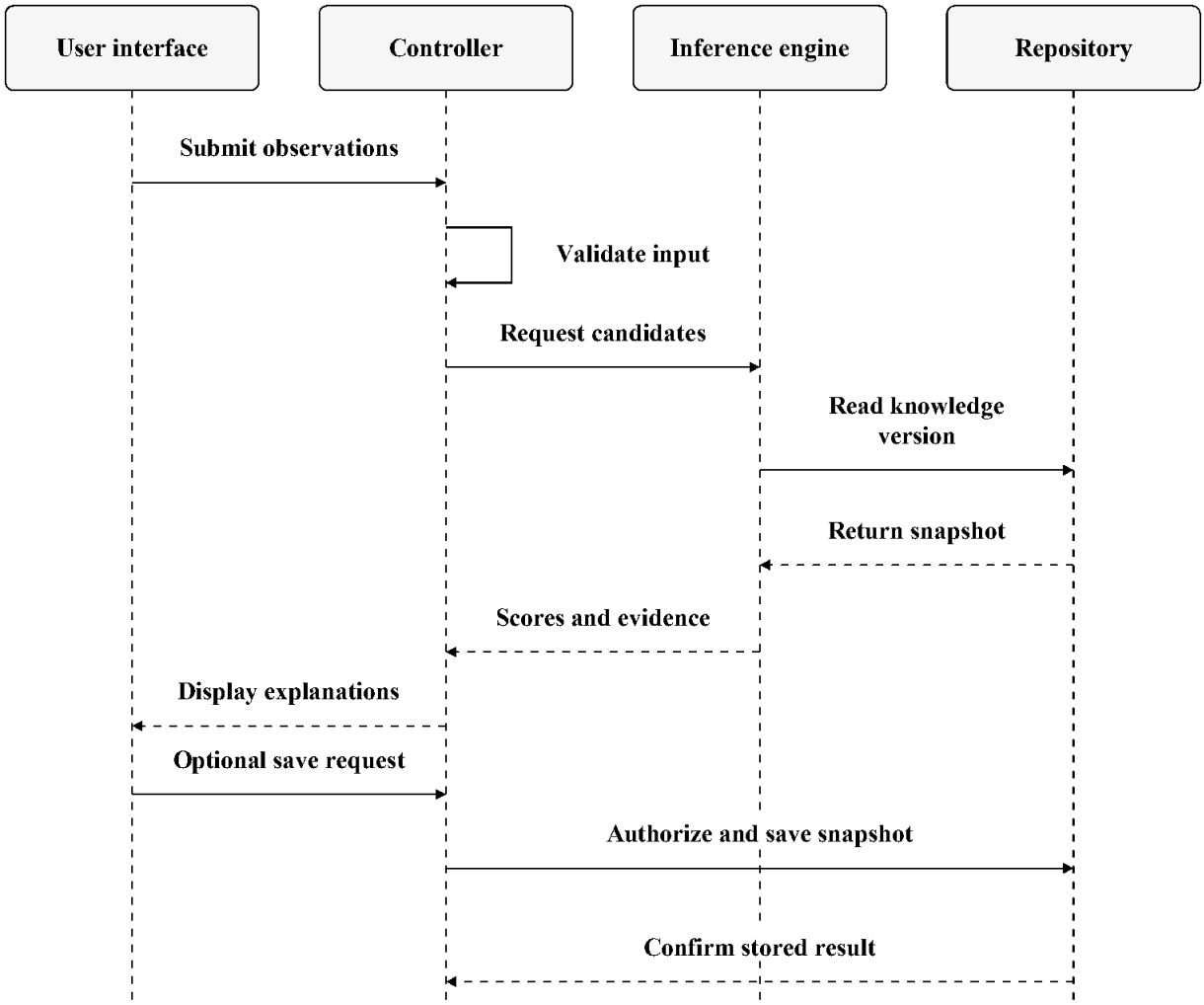


Figure 3.5 MEDIXAI consultation sequence diagram

Figure 3.5 shows the controller validating input before invoking the engine. The repository supplies one knowledge version, and the engine returns candidates with evidence. Saving follows ownership and consent checks; a failed save must not be presented as a successfully stored report.

The sequence is a design specification for the web version. It does not imply that the authentication, persistence or export routes have been implemented.

Consistency across the request

The knowledge read should return both the card data and its version identifier. The engine must use that returned snapshot throughout the calculation, even if an administrator publishes a newer version before the response is displayed.

A save response should identify the stored consultation only after the transaction succeeds.

3.3 DATA FLOW AND UML DIAGRAMS

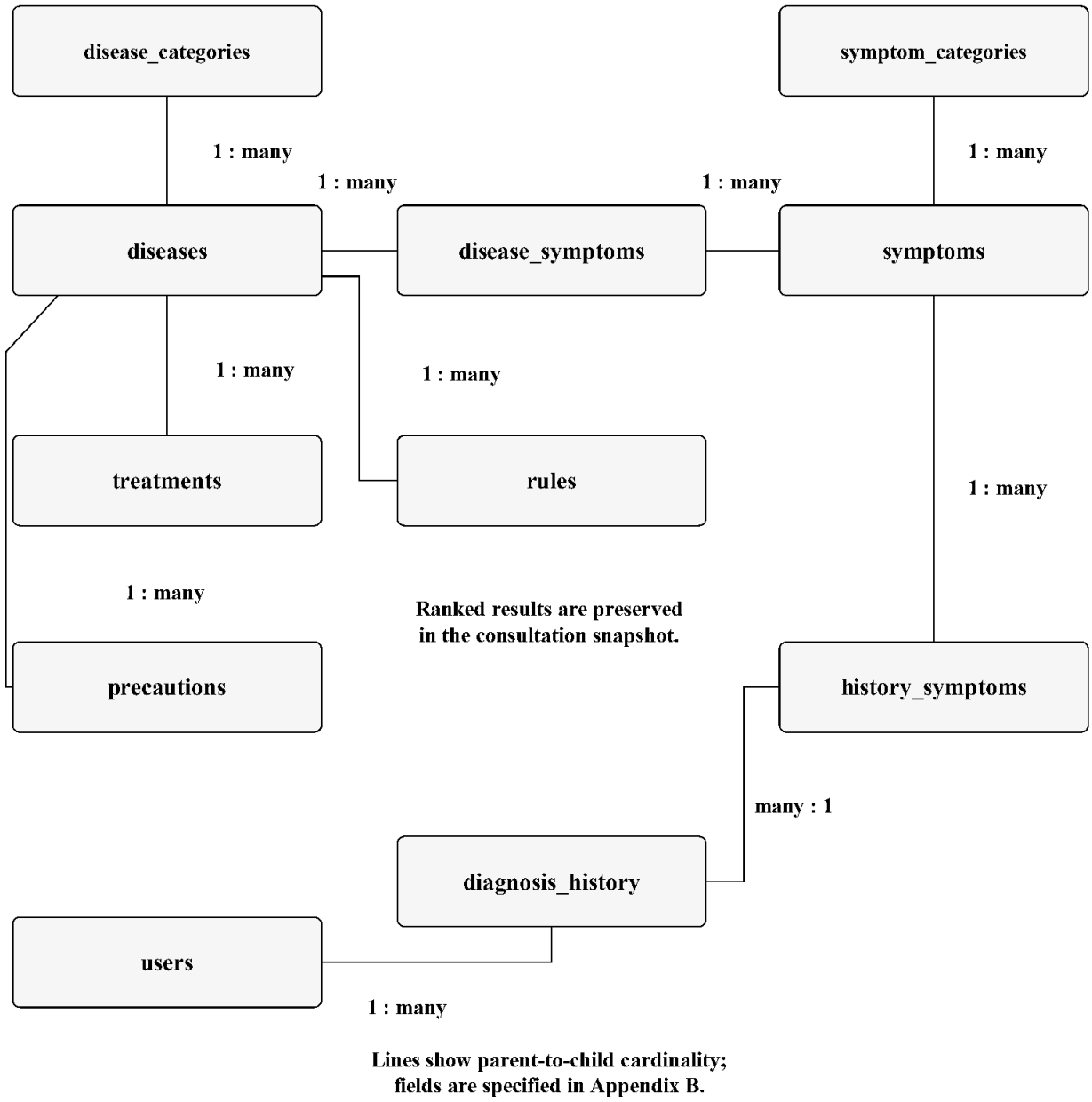


Figure 3.6 MEDIXAI entity relationship overview

Figure 3.6 connects diseases and symptoms through disease_symptoms and records consultation observations in history_symptoms. A result snapshot in diagnosis_history preserves every displayed candidate and its explanation; storing only the top disease would discard the ranked output.

Foreign keys, uniqueness constraints and reviewed publication states protect the knowledge structure. The diagram summarizes the eleven logical tables; Appendix B specifies proposed field and validation choices.

The junction tables should reject duplicate links. History ownership and knowledge-version information must remain available when interpreting a saved result.

3.4 THREAT MODEL AND ATTACK SURFACE

Assets include symptom observations, account credentials, consultation reports, rule weights and administrative privileges. Relevant actors include an ordinary user, an unauthorized visitor, a compromised account and an administrator making an accidental error. The browser, upload interface, report endpoint and editable knowledge base form the main attack surfaces.

Table 3.5 Threat model

Threat	Affected asset or boundary	Proposed response
Account spoofing	Login and session boundary.	Password hashing, secure sessions and throttled login attempts.
Rule tampering	Knowledge editing and import.	Restricted roles, schema checks and versioned publication.
Information disclosure	History and report identifiers.	Owner checks and nonpublic report storage.
Injection	Database queries and rendered text.	Parameterized SQL and context-appropriate escaping.
Denial of service	Large inputs or expensive inference.	Request limits and bounded inference work.
Untraceable edits	Rule changes and publication.	Audit actor, time and version without raw symptom logs.

Table 3.5 maps account spoofing, rule tampering, disclosure, injection, denial of service and untraceable edits to responses in the proposed web system. Python-file modification and terminal disclosure are more immediate local-prototype risks. Clinical misuse remains an output-design risk addressed by the limits in Chapter 4.

Reviewing an attack path

A useful review follows a request from entry to storage and back to output. For example, a report identifier supplied by the browser must be checked against the authenticated owner before any snapshot is read or rendered. Knowledge publication similarly needs authorization before a validated draft becomes active.

The same exercise should cover failure cases. An oversized request should terminate before expensive work, and a rejected administrative change should leave the published version intact. Error messages should help the user recover without revealing another account, a private report path or unnecessary application internals.

These scenarios convert the threat model into concrete checks that can be repeated against an implemented build.

3.5 SECURITY CONTROLS

Table 3.6 Security controls and verification

Control	Implementation requirement	Verification evidence
Authentication	Store password hashes; rotate the session at login.	Inspect stored credentials and test session transitions.
Authorization	Enforce ownership on history, deletion and export.	A second account cannot read or alter the first account's objects.
Input validation	Accept known identifiers and bounded observation counts.	Unknown, malformed and oversized requests are rejected.
Rule integrity	Review knowledge edits and publish versioned snapshots.	Invalid or unauthorized edits cannot become active.
Data protection	Use HTTPS when hosted; keep reports outside public assets.	Check transport settings and direct report access.
Logging	Record security events without raw health details or secrets.	Inspect logs for sensitive content and access restrictions.

Table 3.6 links security controls to implementation requirements and verification evidence. These are proposed ASVS-informed checks, not a compliance claim. State-changing requests also need CSRF protection, output must be escaped, and development debugging must be disabled when hosted.

For privacy, the default guest path should avoid saving names, ages and symptom histories. Where registered users choose storage, the application needs an explicit retention policy and deletion behavior. Backup access must be restricted, and restoration must be tested before backup reliability is claimed.

Verification and maintenance

Controls should be checked at the boundary where they are enforced. An ownership test should request another account's history directly; an input test should submit a malformed identifier to the server; a publication test should attempt to activate an invalid card. A successful browser interaction alone does not exercise these denied paths.

The project should retain a compact verification record containing the scenario, expected behavior, actual behavior and application version. When a route or data model changes, the affected scenarios can be rerun. This makes the security requirements maintainable and keeps untested design intentions separate from demonstrated controls.

Log entries should be limited to the information needed to diagnose or audit the event, with consultation content excluded unless a justified and authorized workflow requires it.

CHAPTER 4

SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The working knowledge is a manually authored set of disease cards rather than a patient training dataset. The current knowledge base contains 38 disease entries, 104 distinct symptom identifiers and 228 weighted disease-symptom mappings. Each card contains six weighted symptoms. Expansion to 40 disease cards remains a development target rather than the verified count.

Table 4.1 Knowledge-base inventory

Category	Disease cards	Weighted mappings
Respiratory	8	48
Viral	6	36
Bacterial	5	30
Digestive	5	30
Skin	4	24
Neurological	3	18
Urinary	3	18
Miscellaneous	4	24
TOTAL	38	228

Table 4.1 summarizes 38 disease cards and 228 weighted mappings across eight categories. These categories reproduce prototype organization, not a medical ontology. Appendix A lists the disease labels. Weights and required symptoms lack documented clinical calibration; practical health use would require expert review, source provenance and independent validation.

Knowledge review procedure

A proposed review record should identify the card, the reviewer, the knowledge version and the reason for a change. Structural checks can confirm that identifiers are unique and weights are positive, but they cannot establish that a symptom association is medically correct.

Before extending the inventory, the development team should resolve duplicate labels, document aliases and keep incomplete cards inactive. This allows educational coverage to grow without confusing record count with validated quality.

4.2 DATA PREPROCESSING

The prototype trims leading and trailing spaces, converts text to lowercase, replaces spaces and hyphens with underscores, and ignores empty comma- separated entries. Inference converts the observations to a set, so repeated symptoms do not increase the score.

Table 4.2 Preprocessing behavior and planned changes

Input issue	Prototype behavior	Proposed handling
Case and separators	High Fever becomes high_fever.	Keep canonical normalization.
Duplicate symptoms	Set membership prevents double counting.	Deduplicate and display once.
Unknown terms	No matching rule contribution.	Show an error and suggest valid choices.
Related terms	fever and high_fever remain different keys.	Use reviewed relationships; do not infer aliases automatically.
Missing observation	Treated as not selected.	Record present, absent or unknown explicitly.
Contradictory terms	Warns and continues ranking.	Ask for correction before proceeding.

Table 4.2 compares normalization, duplicate handling, unknown states and conflict correction. The interface should submit stable symptom identifiers, which the server validates; browser changes must not introduce unauthorized rules, fields or weights.

The existing diarrhea/constipation pair is a project consistency heuristic, not a universal medical contradiction. Time-dependent observations need context; the proposed model should record one defined consultation window.

Normalization examples and boundaries

Normalization should produce a canonical identifier rather than a new interpretation of the user's statement. A reviewed alias can map to an existing code, while an unrecognized term should be returned for correction. The original display label can remain readable without becoming a second identifier.

Duplicate selections should collapse to one observation before scoring. An explicit negative answer should retain its own state, and a skipped question should remain unknown. Recording these choices in preprocessing makes the later score and missing-evidence explanation easier to reproduce.

4.2.1 ALGORITHMS AND METHODOLOGY

Knowledge representation

For one consultation, each selected symptom acts as a ground proposition. The prototype stores disease metadata and a weighted mapping in a Python dictionary. Its required list specifies conditions that must be present for a hypothesis to pass the implemented backward check. A first-order representation can instead attach observations to a patient variable, but the supplied listing does not implement unification or a general first-order prover.

Example conceptual rule
HasSymptom(p, runny_nose) AND
HasSymptom(p, sneezing)
IMPLIES Candidate(p, Common_Cold)

Candidate means supported by an educational rule, not medically confirmed. Weighted matching and logical entailment serve different purposes: a score measures coverage of a card, while a rule check asks whether specified premises are satisfied. The application should expose both rather than interpret a large score as a proof.

Forward and backward reasoning

Forward chaining starts with known facts and derives consequences from satisfied rules. Backward chaining starts with a goal and checks the premises that support it. In MEDIXAI, the current methods are narrower: the forward_chaining function scans weighted symptom cards, and backward_chaining checks a required-symptom set. Multi-step rule firing, an agenda and a proof graph are proposed extensions.

No weights are learned from data. The score is deterministic for a fixed symptom set, card ordering and knowledge version.

Evidence representation and traceability

A useful working fact record contains a canonical symptom identifier and an observation state. The inference result can then record which facts matched a card, which required facts were missing and which conditions were not evaluated. This keeps input representation separate from presentation text.

For a future multi-step rule engine, the trace should additionally record the rule identifier, the premises used and the newly derived conclusion. Repeated derivations should not create an endless agenda. A stopping condition and a clear distinction between supplied facts and derived facts are therefore part of the proposed methodology.

These trace records support inspection of the reasoning process without claiming that the current weighted matcher already performs formal logical proof.

Weighted matching and ranking

For disease d, let W be the sum of all six symptom weights and M the sum of weights for symptoms present in the input. The prototype first rounds the base score to two decimal places. If any required symptom is missing, it multiplies that rounded score by 0.60 and rounds again .

BaseScore(d) = round(100 * M / W, 2)

FinalScore(d) = BaseScore(d)
when all required symptoms are present

FinalScore(d) = round(0.60 * BaseScore(d), 2)
when at least one required symptom is missing

Only cards with at least one matched symptom enter the ranking. Results are sorted in descending score order, and the command-line output prints at most ten. Ties retain the dictionary order because Python sorting is stable; the proposed application should document a stable secondary key or display ties explicitly.

The penalty is applied once whether one or several required symptoms are missing. It is a heuristic project choice, not a medically established likelihood adjustment. Unrelated extra symptoms do not reduce the score, so selecting more symptoms can make many cards appear strongly supported.

Scores need not sum to 100 across diseases. A 100 percent score means that all weighted symptoms on one card were selected. It does not mean 100 percent diagnostic accuracy or certainty. The interface should label the value as a rule- match score.

Interpreting the score

The denominator belongs to the candidate card, not to the number of observations entered by the user. Two candidates can therefore receive different percentages from overlapping evidence because their total weights differ. A displayed explanation should show both matched weight and total weight so that this difference is visible.

The required-symptom penalty is applied once when the required set is incomplete. The implementation should record the missing set and the applied factor explicitly. It should not repeatedly multiply the score for each missing symptom unless a separately specified algorithm intentionally changes that behavior.

Stable sorting also needs a documented tie policy. Retaining stored order makes the prototype reproducible for a fixed dictionary, but that order should not be presented as additional evidence. A proposed interface can show tied scores clearly and let the user compare their supporting observations.

Backward verification and consistency checking

The expanded backward method examines the highest-ranked candidate and returns the required symptoms absent from the input. It does not recursively derive subgoals and does not ask interactive follow-up questions. The earlier five-disease example does ask one combined question, but that behavior should not be attributed to the expanded listing.

A proposed follow-up loop should ask about one unresolved requirement at a time, accept present/absent/unknown answers, and recompute the ranking after each response. It should stop when the user declines, the evidence is sufficient for the rule check, or a configured question limit is reached.

Resolution and contradiction handling

The prototype detects three specified pairs: fever with no_fever, high_fever with no_fever, and diarrhea with constipation. This is pairwise consistency checking. Full propositional resolution requires clauses and a refutation procedure, as illustrated below.

Illustrative Horn rule: A AND B IMPLIES C
Clause form: NOT A OR NOT B OR C
Facts: A, B
Negated query: NOT C

Resolve with A: NOT B OR C
Resolve with B: C
Resolve with NOT C: empty clause

This trace demonstrates entailment in the toy rule system. It is a proposed teaching example, not an executed capability of the medical prototype and not a proof of a medical diagnosis.

Unknown evidence and bounded reasoning

A missing requirement and an explicitly absent requirement are different observations. A focused follow-up question should preserve that distinction and allow the user to decline. Recalculation should use the updated observation set while retaining the same knowledge version for the consultation.

A formal resolution exercise should retain each derived clause and its parent clauses so that an examiner can follow the refutation. Repeated clauses should be detected, and a resource limit should terminate the demonstration predictably. These requirements concern the proposed teaching module, which remains separate from pairwise conflict checks in the prototype.

Testing should verify both a successful derivation and an inconclusive case. The output must state which reasoning procedure actually ran, rather than use a general label that hides differences between matching, verification and proof.

4.2.2 RESULTS AND DISCUSSION

Worked scoring examples

The following values were recalculated from the expanded dictionary using a faithful reconstruction of its normalization, matching, rounding and penalty rules. They are deterministic software examples, not patient outcomes, clinical tests or execution evidence for the proposed web application.

Table 4.3 Worked candidate scores

Input and candidate	Matched / total weight	Penalty	Score
runny_nose + sneezing Common Cold	19 / 43	None	44.19
runny_nose + sneezing Allergic Rhinitis	19 / 45	None	42.22
runny_nose + dry_cough Common Cold	18 / 43	x 0.60	25.12
high_fever + body_pain + headache + rash Dengue Fever	34 / 46	None	73.91
Same four symptoms Influenza	24 / 45	x 0.60	32.00

In Table 4.3, Common Cold's required symptoms contribute 10 + 9 = 19 out of 43, producing 44.19. When sneezing is missing, round(100 x 18 / 43, 2) = 41.86; the penalty gives round(41.86 x 0.60, 2) = 25.12.

The close Common Cold and Allergic Rhinitis scores illustrate overlapping rules. The system should show the evidence for both candidates and request missing information instead of claiming that the higher score is medically correct.

Reproducing a worked example

A reproducible example should retain the selected symptom identifiers, the candidate card, each matched weight, the total card weight and the rounding order. The reviewer can then calculate the unpenalized percentage before checking whether the required set triggers the factor.

When examples are moved into automated tests, expected values should remain fixed and readable. Any later change to a card should be reviewed separately from a change to the scoring procedure so that a changed result has an identifiable cause.

Deterministic behavior checks

Table 4.4 Reconstructed logic checks

Case	Observation	Interpretation
Complete Cold card	Common Cold scores 100.00.	All six card symptoms are matched.
Two required Cold symptoms	Cold 44.19; Allergic Rhinitis 42.22.	Multiple rules can have support.
One requirement missing	Cold scores 25.12 for runny_nose and dry_cough.	Single 0.60 penalty is applied.
Four-symptom example	Dengue 73.91; Influenza 32.00.	Weights and requirements affect order.
Repeated symptom	Repeated runny_nose leaves 44.19 unchanged.	Set membership prevents double counting.
Unknown term	No candidates for unlisted_symptom.	No supported dictionary match.
Empty input	No candidates.	Empty evidence has no matches.
fever + no_fever	Pair is flagged; ranking logic still produces candidates.	Warning does not stop the prototype.

Table 4.4 checks complete and partial evidence, penalties, duplicates, unknown terms, empty input and conflicts. The reconstructed calculations establish software behavior. Inspection of diagnose() shows no early return after a contradiction warning. No performance timings or clinical accuracy measurements were supplied.

The no-match path returns before the final academic disclaimer in the listing. The planned application should place its educational-use notice consistently on every result state, including errors and empty results.

Extending the behavior checks

Additional controlled cases should cover a score tie, multiple missing required symptoms and a knowledge card rejected during loading. The tie case checks ordering; the missing-evidence case checks that the penalty is applied once; the invalid-card case checks that malformed knowledge cannot reach the calculation.

A useful test record includes the complete input and expected evidence, not only the top candidate. This detects regressions in missing-symptom lists and explanations even when the first-ranked disease happens to remain unchanged.

Evaluation plan and limitations

Table 4.5 defines planned checks for correctness, explanation fidelity, robustness, performance, security and usability. Compare exact matching, unweighted overlap and weighted ranking using the same knowledge base and inputs; this separates the algorithm effect from changes in the disease list.

Table 4.5 Planned evaluation measures

Measure	How it will be evaluated
Functional correctness	Expected score, ranking, missing evidence and conflict status for controlled cases.
Explanation fidelity	Recompute each displayed score from the evidence shown to the user.
Robustness	Unknown identifiers, duplicates, contradictory input and maximum-size requests.
Performance	Report machine configuration, knowledge size, repetitions and latency distribution.
Security	Ownership isolation, unauthorized edits, injection attempts and report access.
Usability	Observe whether students can explain a result and identify missing evidence.

Diagnostic accuracy, precision and recall would require an independently labelled, appropriately reviewed clinical evaluation set. They cannot be derived from rule-match percentages. No such dataset or measurements are available for this project.

Other limitations include hand-selected weights, incomplete vocabulary, no temporal context, no calibrated uncertainty and static emergency metadata. In particular, a false emergency flag does not establish that a user is safe.

Evaluation records and reporting

Each comparison should retain its knowledge version, fixed case set and expected outputs. Performance measurements should state whether they cover only the inference function or the entire request, because validation, storage and PDF generation introduce separate work.

Usability tasks should ask participants to interpret a score, identify a missing requirement and recognize a no-match result. Observations from those tasks can guide wording and layout changes. They should be reported as evidence about interaction with the educational system, with the sample and procedure described clearly.

CHAPTER 5

CONCLUSION

MEDIXAI provides a concrete setting for studying knowledge-based AI through explicit symptom facts, weighted disease cards and inspectable reasoning. The design defines the functional scope, proposed application architecture, module interfaces, data model and security requirements. The expanded prototype supplies 38 disease cards with 228 weighted mappings and demonstrates deterministic candidate ranking.

The worked examples show how required-symptom penalties change a score and how overlapping evidence produces several candidates. They also reveal why match coverage must be separated from medical accuracy. The current forward and backward methods are limited to card evaluation and required-set checking, while pairwise contradiction detection remains distinct from a full resolution prover.

The next development stage should first validate the knowledge schema and provenance, implement an inference interface with evidence traces, and add controlled tests. The database and application routes can then be integrated with ownership checks, private history and consistent report generation. A later evaluation should compare reasoning variants using fixed inputs and review the usability of explanations.

Future extensions include explicit negative and unknown observations, focused follow-up questions, multi-step Horn-rule reasoning and a bounded resolution demonstration. Cross-domain reuse should be tested with a second, nonmedical knowledge base. Clinical use would require a separate body of expert review and validation; it is not an outcome established by this academic project.

Development priorities

The immediate priority is an implementation that preserves the report's evidence contract from input to export. Canonical observations, a fixed knowledge version and structured candidate records provide a common foundation for the interface, database and test suite. Completing these elements first makes later feature work easier to evaluate.

The next priority is repeatable verification. The arithmetic examples can become controlled regression cases, while the proposed acceptance scenarios can exercise input correction, ownership, publication and export consistency. Results should be recorded only after execution against an identified build.

The project's academic value lies in showing how knowledge, inference and explanation fit together in a system that can be inspected.

REFERENCES

[1] MEDIXAI Development Group. (2026). *Project Planning and Database Architecture Specifications.* Internal design logs and feature roadmap.

[2] Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th US ed.). Pearson. https://aima.cs.berkeley.edu/

[3] Buchanan, B. G., & Shortliffe, E. H. (Eds.). (1984). *Rule-based expert systems: The MYCIN experiments of the Stanford Heuristic Programming Project.* Addison-Wesley. https://www.shortliffe.net/Buchanan-Shortliffe-1984/MYCIN%20Book.htm

[4] van Melle, W., Shortliffe, E. H., & Buchanan, B. G. (1984). EMYCIN: A knowledge engineer's tool for constructing rule-based expert systems. In B. G. Buchanan & E. H. Shortliffe (Eds.), *Rule-based expert systems: The MYCIN experiments of the Stanford Heuristic Programming Project* (pp. 302-313). Addison-Wesley. https://people.dbmi.columbia.edu/~ehs7001/Buchanan-Shortliffe-1984/Chapter-15.pdf

[5] OWASP Foundation. (2025). *Application Security Verification Standard* (Version 5.0.0). https://owasp.org/projects/asvs

[6] United Nations Department of Economic and Social Affairs. (n.d.). *Goal 3: Good health and well-being* (Target 3.8). Retrieved September 27, 2026, from https://sdgs.un.org/goals/goal3

[7] United Nations Department of Economic and Social Affairs. (n.d.). *Goal 4: Quality education* (Target 4.4). Retrieved September 27, 2026, from https://sdgs.un.org/goals/goal4

[8] Forgy, C. L. (1982). Rete: A fast algorithm for the many pattern/many object pattern match problem. *Artificial Intelligence, 19*(1), 17-37. https://doi.org/10.1016/0004-3702(82)90020-0

[9] Secret Society Software, LLC. (2025). *CLIPS reference manual: Volume I, Basic programming guide* (Version 6.4.2). https://www.clipsrules.net/documentation/v642/bpg642.pdf

[10] Phillips, P. J., Hahn, C. A., Fontana, P. C., Yates, A. N., Greene, K. K., Broniatowski, D. A., & Przybocki, M. A. (2021). *Four principles of explainable artificial intelligence* (NISTIR 8312). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.IR.8312

APPLICATION SCREENSHOTS

The full-stack browser application presents symptom collection, expandable symptom analytics, dynamic knowledge-base CRUD forms, persistent SQLite evaluation logs, and transparent rule-based reasoning in one dark-mode single-page workspace (index.html connected to app.py and medixai.db).

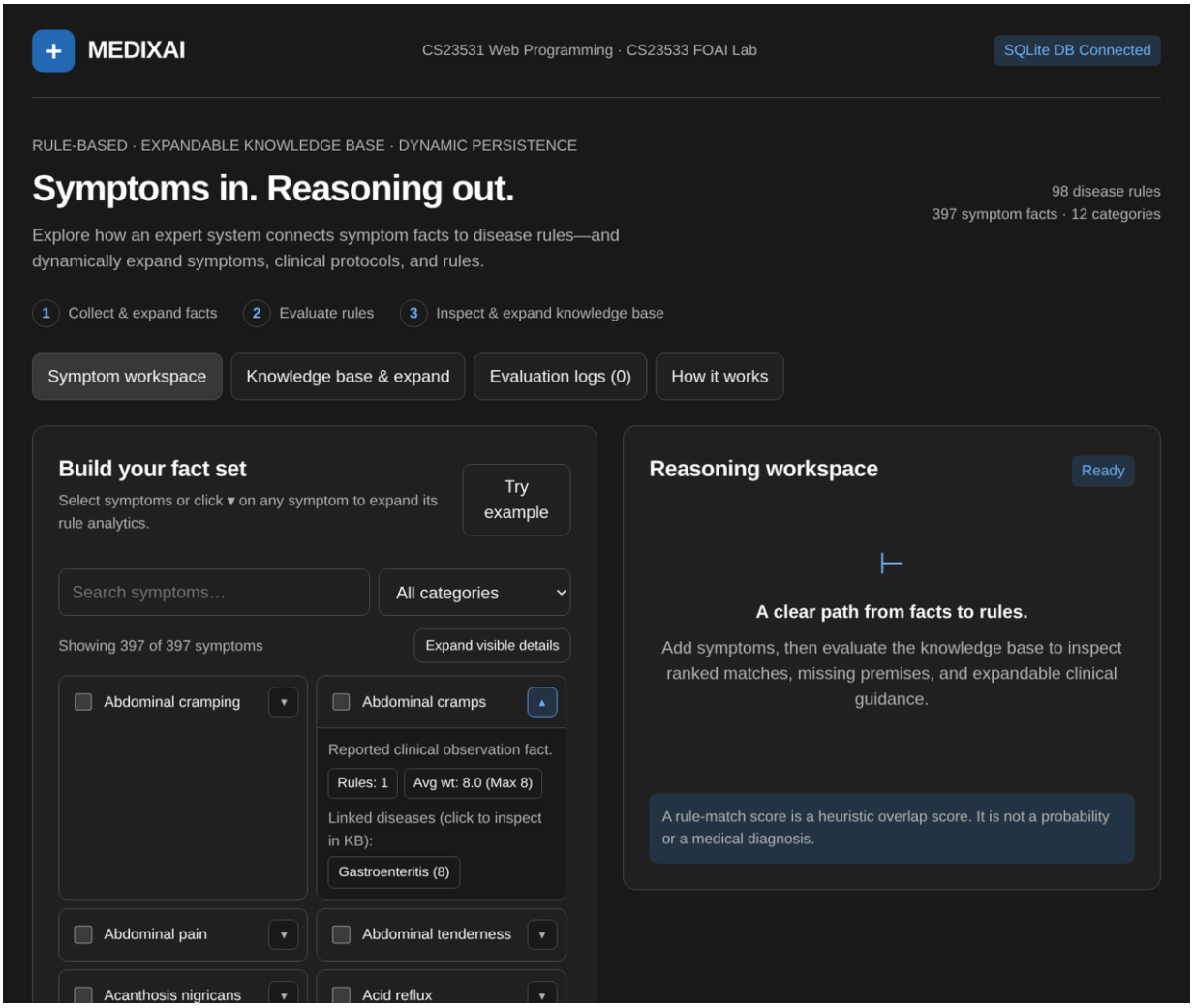


Figure S.1 MEDIXAI symptom workspace

Figure S.1 shows the MEDIXAI symptom workspace connected to the SQLite database (98 disease rules, 397 symptom facts, 12 categories). Search and category controls filter symptoms live, while clicking the disclosure toggle on any symptom expands its linked disease rules, average/maximum weight, and safety metadata.

APPLICATION SCREENSHOTS CONTINUED

This screen records a consultation after four facts have been selected. The interface places the active observations beside ranked candidate cards, allowing the user to compare the input with the evidence supporting a selected result.

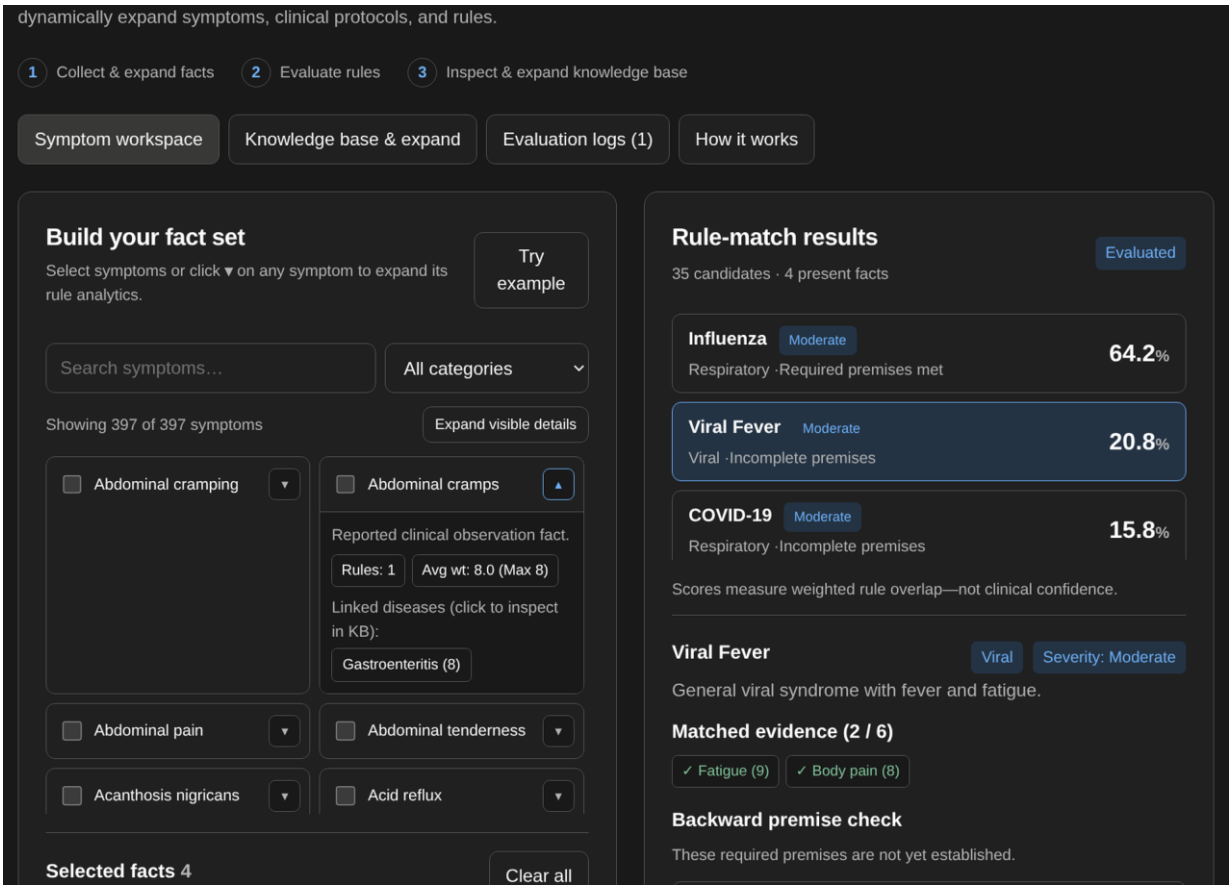


Figure S.2 MEDIXAI rule-match results and premise check

Figure S.2 shows the evaluated rule-match results and interactive backward premise check after selecting clinical facts. Ranked candidates display severity badges and weighted overlap scores, while the detail view exposes matched evidence, Yes/No/Unsure buttons for missing required premises, and expandable clinical treatment and precaution guidance.

APPLICATION SCREENSHOTS CONTINUED

The Knowledge base & expand tab provides both an interactive browser for all 98 disease rules and an expandable CRUD form drawer that persists new disease rules, weighted symptoms, and contradiction pairs directly into the SQLite database via RESTful POST endpoints.

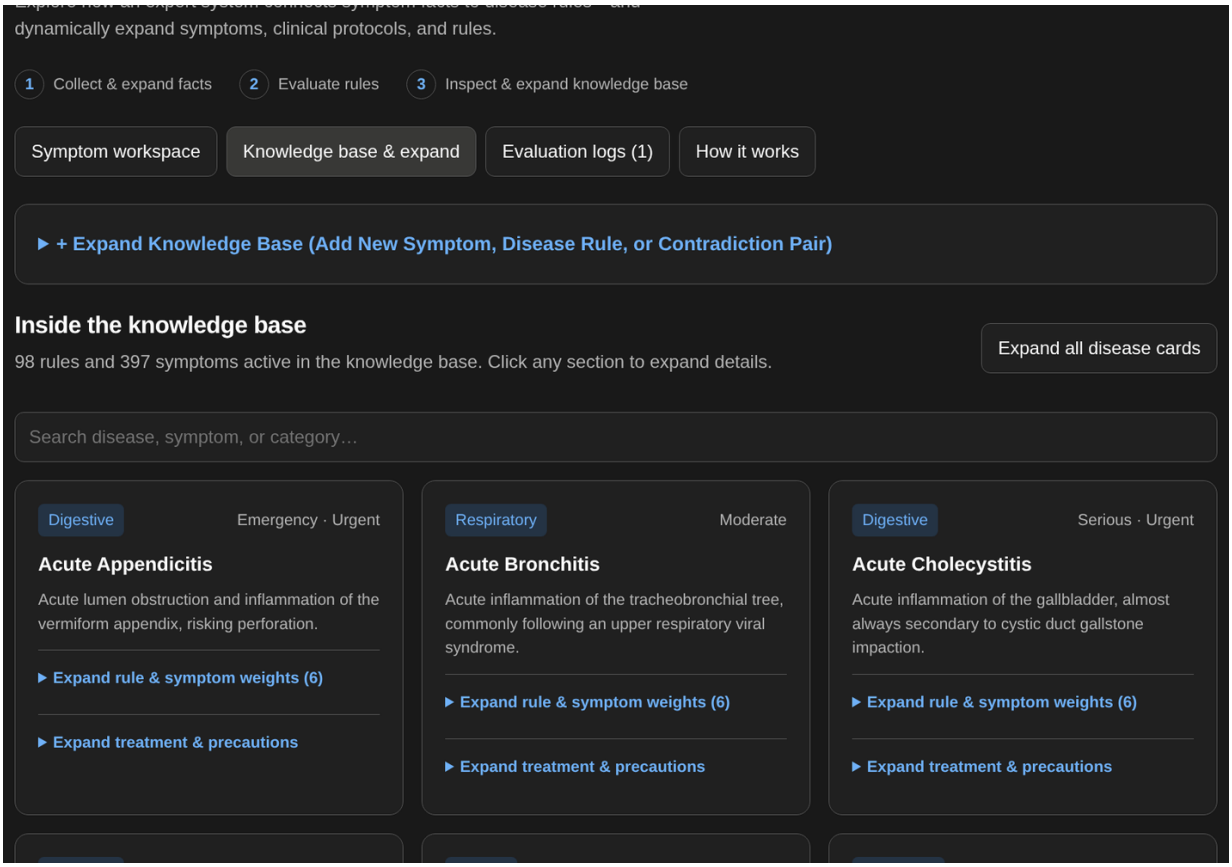


Figure S.3 MEDIXAI knowledge-base browser

Figure S.3 shows the Knowledge base & expand view with rule cards displaying category badges, severity levels, expandable symptom weights, supportive treatments, and a one-click button to load required premises directly into the reasoning workspace.

APPLICATION SCREENSHOTS CONTINUED

The explanation view documents how the browser demonstration treats facts, rule matches and missing premises.

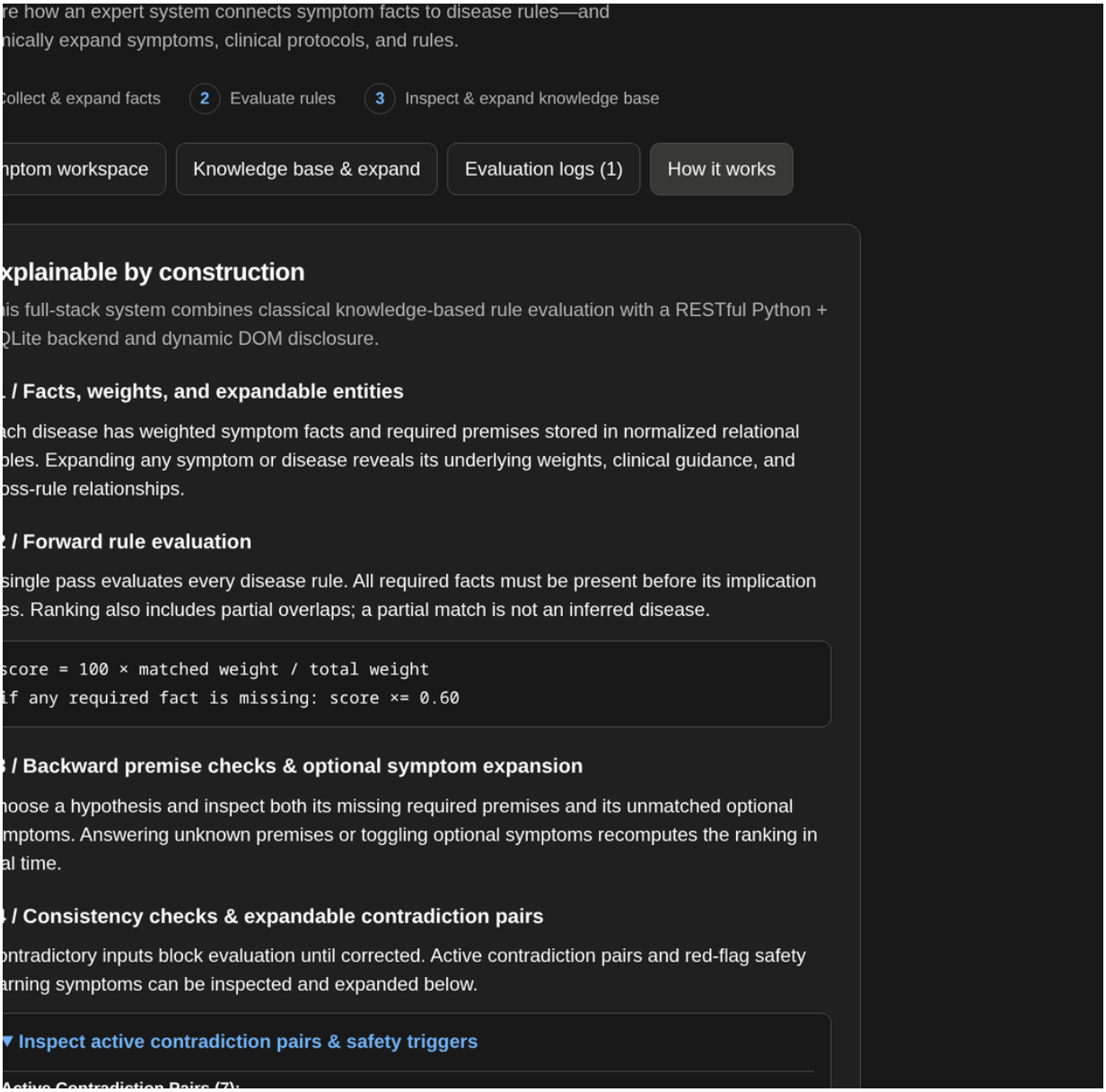


Figure S.4 MEDIXAI reasoning methods and safety information

Figure S.4 explains the four-stage reasoning pipeline: weighted symptom facts, forward rule evaluation with the 0.60 missing-premise multiplier, interactive backward premise checks, and active contradiction pairs alongside red-flag triage warning triggers.

APPENDIX A

KNOWLEDGE BASE INVENTORY

Table A.1 lists entries 1-19 from the 38-card knowledge base, grouped by category. The entry numbers are report indices, not database identifiers. The categories reproduce the prototype labels.

Table A.1 Disease inventory

No.	Disease label	Category
1	Common Cold	Respiratory
2	Influenza	Respiratory
3	COVID-19	Respiratory
4	Pneumonia	Respiratory
5	Asthma	Respiratory
6	Acute Bronchitis	Respiratory
7	Sinusitis	Respiratory
8	Allergic Rhinitis	Respiratory
9	Dengue Fever	Viral
10	Chickenpox	Viral
11	Typhoid Fever	Bacterial
12	Tuberculosis	Bacterial
13	Food Poisoning	Digestive
14	Gastritis	Digestive
15	Eczema	Skin
16	Migraine	Neurological
17	Urinary Tract Infection	Urinary
18	Kidney Stones	Urinary
19	Heat Stroke	Miscellaneous

The source listing contains 38 entries. The planned expansion target is 40 disease cards.

Use stable identifiers when importing these labels. A display-name correction should not create a second card or break the link to an earlier saved consultation. Keep the imported inventory tied to its knowledge version.

APPENDIX A

KNOWLEDGE BASE INVENTORY

Table A.2 Disease inventory continued

No.	Disease label	Category
20	Dehydration	Miscellaneous
21	Measles	Viral
22	Mumps	Viral
23	Hand Foot and Mouth Disease	Viral
24	Viral Fever	Viral
25	Cholera	Bacterial
26	Whooping Cough	Bacterial
27	Bacterial Meningitis	Bacterial
28	Gastroenteritis	Digestive
29	GERD	Digestive
30	Peptic Ulcer	Digestive
31	Ringworm	Skin
32	Scabies	Skin
33	Psoriasis	Skin
34	Tension Headache	Neurological
35	Vertigo	Neurological
36	Cystitis	Urinary
37	Iron Deficiency Anemia	Miscellaneous
38	Heat Exhaustion	Miscellaneous

Table A.2 lists entries 20-38, each with six weighted mappings. The inventory shows project coverage, not validated associations, severity or guidance.

Coverage and review status

Category labels support organization and browsing; they do not change the scoring formula. The loader should preserve a clear mapping from each disease card to its six symptom weights and required-symptom set.

Any new card should be structurally validated and assigned a review status before publication.

APPENDIX B

PROPOSED DATA DICTIONARY

Table B.1 Knowledge data dictionary

Table	Key fields	Constraints and notes
disease_categories	category_id, name	Integer primary key; unique name.
symptom_categories	category_id, name	Integer primary key; unique name.
diseases	disease_id, category_id, name, description, severity, emergency	Foreign key to category; unique canonical name; reviewed status and source metadata proposed.
symptoms	symptom_id, category_id, code, display_name	Unique canonical code; reviewed aliases managed explicitly.
disease_symptoms	disease_id, symptom_id, weight, required	Composite primary key; both foreign keys; positive bounded weight; required is 0 or 1.
rules	rule_id, disease_id, premises, conclusion, priority, version	Validated structured conditions; inactive until reviewed; never executable Python text.

Table B.1 defines knowledge records, keys and validation constraints. Rules need structured conditions; a disease identifier and score threshold do not encode an arbitrary logical rule. Required and optional symptom sets can support the first implementation, with complex clauses added after their evaluation semantics are defined.

All six current card weights are positive. The loader must reject an empty or zero-total card to prevent division by zero.

Publication checks

Before a knowledge version becomes active, validate every foreign key, symptom code, required flag and scoring denominator. All required symptoms should also occur in the card's weighted mapping so that the explanation can account for them.

A rejected draft should produce a reviewable error list and leave the last published snapshot available. Successful publication should identify the exact version used by subsequent consultations.

APPENDIX B

PROPOSED DATA DICTIONARY

Table B.2 Application data dictionary

Table	Key fields	Constraints and notes
users	user_id, username, password_hash, role	Unique username; no plaintext password; limited roles.
treatments	treatment_id, disease_id, guidance_text, review_status	Reviewed educational information; automatic dosage generation excluded.
precautions	precaution_id, disease_id, text, review_status	Foreign key; validated text and publication status.
diagnosis_history	history_id, user_id, created_at, knowledge_version, result_snapshot	User ownership; immutable result snapshot; retention and deletion policy.
history_symptoms	history_id, symptom_id, observation_state	Composite primary key; present, absent or unknown; consistent consultation context.

Table B.2 defines account, guidance and consultation-history records with their constraints. A result snapshot preserves the ranked list, weights and explanation evidence. Its knowledge_version supports reproducibility after rule changes. A normalized result table could be added if reporting needs justify expanding the eleven-table design.

Foreign-key checks must be enabled for each SQLite connection. Administrative deletion should not silently remove information needed to interpret old results; deactivation or versioned replacement is preferable. Guest results should remain transient unless the user explicitly chooses a storage workflow.

Storage and retrieval checks

The history write should store the result and its observation rows atomically. Retrieval should check ownership before returning either the observation list or the snapshot, including when the request is made through an export route.

Retention and deletion behavior should be specified before personal histories are enabled. Tests should confirm that deleting one account's consultation does not alter shared knowledge records or another account's data.

The stored snapshot should retain the evidence needed to explain the original ranking, even when display labels or active knowledge cards change later.

APPENDIX C

INFERENCE PSEUDOCODE

C.1 Prototype scoring behavior

```
INPUT: symptom_text, knowledge_base

S = set(normalize(nonempty comma-separated tokens))
results = []

FOR each disease d in stored order:
    matched = keys(d.weights) intersect S
    IF matched is empty:
        CONTINUE

    M = sum(d.weights[s] for s in matched)
    W = sum(all values in d.weights)
    score = round(100 * M / W, 2)
    missing_required = d.required minus S

    IF missing_required is not empty:
        score = round(0.60 * score, 2)

    append disease, score, matched,
            missing_required to results

STABLE SORT results by descending score
RETURN results
```

The pseudocode describes the expanded scoring method without reproducing medical guidance. The interface prints the first ten ranked candidates, verifies the required symptoms of the first candidate and then explains that candidate. The proposed implementation should return this information in a structured record.

For D cards, at most K symptoms per card and U submitted symptoms, set construction costs O(U), card scanning costs O(DK), and sorting costs O(D log D), under ordinary hash-set assumptions. These are analytical bounds, not benchmark results.

Implementation notes

Keep the rounding order identical across the reusable engine and its regression examples. A required-symptom penalty changes the already rounded percentage in the prototype, so delaying all rounding until display can produce a different result.

The returned candidate should carry enough evidence to regenerate the explanation without rerunning the calculation. This also supports a direct comparison between a saved snapshot and its exported representation.

APPENDIX C

INFERENCE PSEUDOCODE

C.2 Proposed controlled inference workflow

```
INPUT: validated observation identifiers

load one reviewed knowledge snapshot
reject malformed or unknown observations

conflicts = check_contextual_conflicts(observations)
IF conflicts exist:
    RETURN correction request

candidates = score_cards(observations, snapshot)
IF candidates are empty:
    RETURN no-match result with educational notice

FOR each displayed candidate:
    verify required evidence
    construct explanation from the same result data

IF user accepts follow-up and unknown evidence exists:
    ask one bounded question
    update observation state and recompute

DISPLAY candidates, explanations and knowledge version

IF user requests saving:
    authorize ownership and store result snapshot
```

This workflow is a proposed extension. It introduces an input correction gate, explicit unknown states and consistent output notices. A separate Horn-rule module may later maintain an agenda and record each fired rule. That addition must be evaluated independently of the existing weighted matcher.

A resolution demonstration should use finite clauses, a defined inference limit and a proof trace. Failure to find a refutation within a resource limit must be reported as inconclusive rather than false.

Workflow outcomes

Define separate result types for correction required, no match, ranked output and storage failure. Each type should identify the next available action so that the interface does not represent an incomplete request as a successful consultation.

The save operation should use the result already computed for the current knowledge version. It should not silently recalculate against newly published cards while presenting the earlier explanation.

APPENDIX D

IMPLEMENTATION AND ACCEPTANCE PLAN

Table D.1 Planned acceptance tests

ID	Scenario	Acceptance condition
T01	Canonical and duplicate symptoms	Equivalent selections yield identical scores.
T02	Missing required evidence	Penalty and missing list agree with the card.
T03	Conflicting observations	Web workflow requests correction before ranking.
T04	Unknown symptom identifier	Server rejects the identifier with a clear error.
T05	Cross-account history access	Request is denied without exposing consultation data.
T06	Unauthorized knowledge edit	Request is denied and recorded as a security event.
T07	Malformed or zero-weight card	Publication is rejected before inference.
T08	Score explanation and PDF	UI, stored snapshot and export show the same evidence.
T09	Knowledge update during a session	Consultation uses one consistent version.
T10	No match or failed request	No disease certainty is asserted; notice remains visible.

Table D.1 states planned application acceptance criteria, distinct from the reconstructed arithmetic examples in Chapter 4. These scenarios have not been reported as passed. Each executed test should record its input, expected output, actual result and tested software version.

Acceptance evidence record

For each executed scenario, record the test identifier, input, expected result, actual result and software version. Include the knowledge version whenever the scenario depends on scoring. A failed case should retain enough evidence to reproduce the problem.

Repeat related cases after a fix. A change to export formatting, for example, should still preserve the evidence checked by T08, while a change to publication must preserve the consistent consultation behavior checked by T09.

Keep the test evidence with the reviewed application build.

APPENDIX D

IMPLEMENTATION AND ACCEPTANCE PLAN

Table D.2 Proposed development milestones

Stage	Deliverable	Exit condition
1 Knowledge review	Canonical cards and provenance fields.	All records validate; gaps and review status are visible.
2 Inference core	Reusable engine and explanation schema.	Deterministic logic tests pass.
3 Persistence	Eleven-table database and repository.	Relationships, snapshots and rollback are tested.
4 Application	Guest workflow, login, results and history.	Ownership and input controls pass acceptance tests.
5 Administration and export	Reviewed knowledge editing and PDF reports.	Unauthorized edits fail; outputs remain consistent.
6 Evaluation	Comparative tests and final report update.	Measured results replace planned evaluation claims.

Table D.2 orders development milestones by dependency and defines their exit conditions. Knowledge validation precedes integration so that interface work does not conceal reasoning defects. Authentication and ownership checks must be completed before personal consultations are stored.